Lab 12: Contrast

Section 1: Transformers: Attention is all you need!

Transformers process input sequences in parallel, without considering the inherent order of tokens (unlike RNNs and LSTMs, which process tokens sequentially). This means in addition to the token itself, the model needs a way to understand the position of each token in the sequence, which is where positional encodings come in.

Given the input sentence "I love pizza", we will walk through how it is encoded using the self-attention mechanism.

The embeddings of each token $Embed_{token}$ is learned prior to model training and can simply be acquired through a lookup table. Part of the learned table of is shown below:

$Token_{ID}$	$Token$	$Embed_{token}$
0	the	$[0.54 \;\; 0.36 \;\; 0.97]$
.....	.....	.....
39	love	$[0 \;\; 1 \;\; 0]$
.....	.....	.....
224	pizza	$[0 \;\; 0 \;\; 1]$
.....	.....	.....
999	I	$[1 \;\; 0 \;\; 0]$
.....	.....	.....

Now let's apply a simple rule-based algorithm to create the positional encoding $\text{PosEnc}_{i,j}$ for a sequence of length 3 with a (postional) embedding dimension of 3.

i represents the position of the token in the sequence, while j represents the index within the token's positional embedding vector we're constructing:

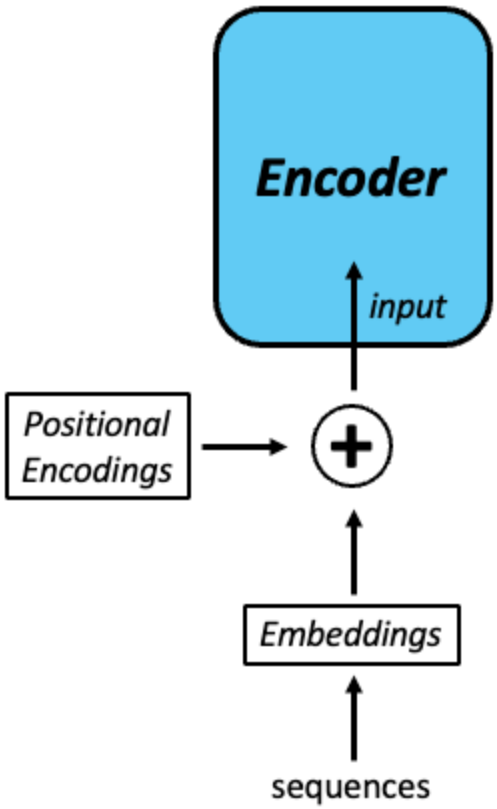
- For even indices (i) in the sequence: $\text{PosEnc}_{i,j} = i + j$
- For odd indices (i) in the sequence: $\text{PosEnc}_{i,j} = i - j$

❓ 1.1 Fill up the positional encoding table below

✅ Answer:

Position_i	$\text{PosEnc}_{i,0}$	$\text{PosEnc}_{i,1}$	$\text{PosEnc}_{i,2}$
0	0	1	2
1	1	0	-1
2	2	3	4

❓ 1.2 Using the structure below, determine the input for each token $Input_{token}$ to the encoder (highlight in blue) using the elementwise addition of the vectors ⊕.



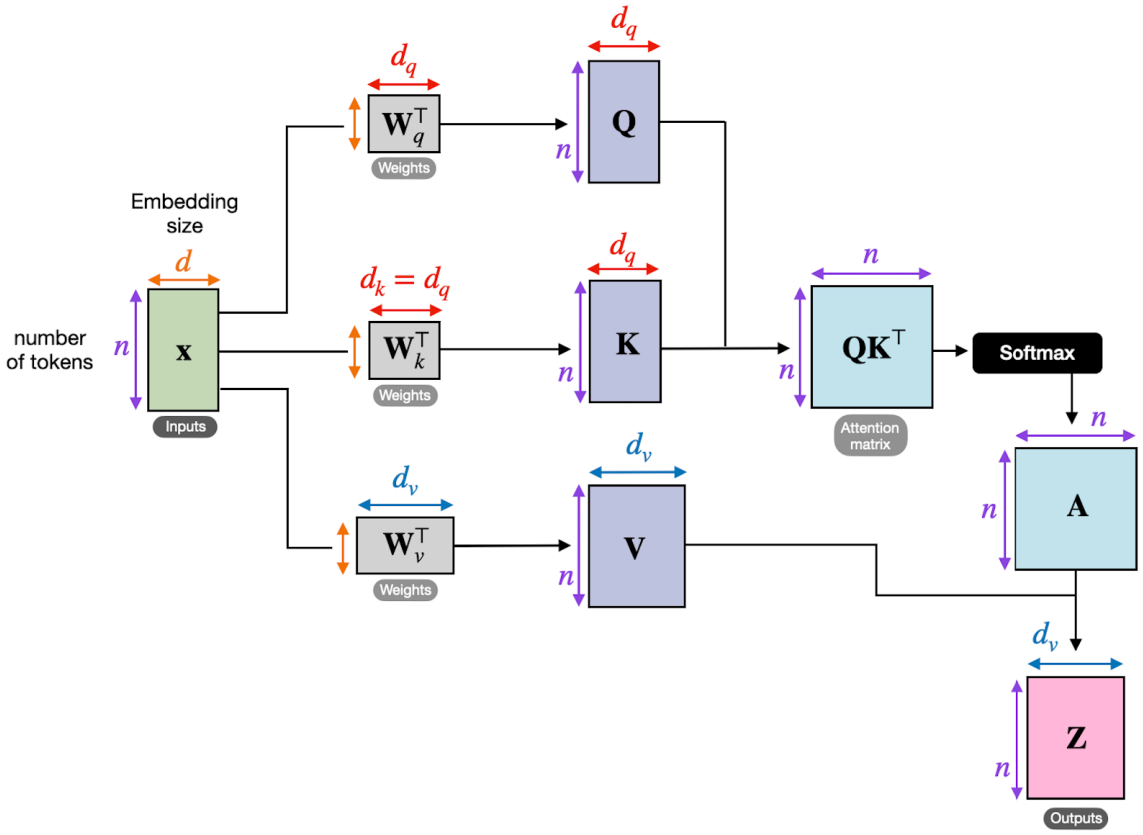
✅ $Input_{\text{I}}$ = [1, 1, 2]

✅ $Input_{\text{love}}$ =[1, 1, -1]

✅ $Input_{\text{pizza}}$ = [2, 3, 5]

Congratulations! You've successfully calculated the input to the **self-attention** unit, which will subsequently contextualize these input embeddings through "attention". Let's take a closer look at the architecture of the self-attention unit (displayed below) and explore what's happening inside this neural network component.

To avoid confusion, we will consistently use the terms from the diagram below to refer to the different elements and representations in the self-attention unit.



The weight matrices for the queries (W_q), keys (W_k), and values (W_v) are here pre-defined for our example as:

- W_q = W_k = W_v =

$$\begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix}$$

❓ 1.3 Compute the **query** Q, **key** K and **value** V vectors for each token using the given weight matrices (i.e., identity matrices) above.

Note: $Attention\ Vector_{token} = x_{token} \cdot W^T_{attention\_representation}$

✅ Query vectors:

- Q_{I} = [1, 1, 2]
- Q_{love} = [1, 1, -1]
- Q_{pizza} = [2, 3, 5]

✅ Key vectors:

- K_{I} = [1, 1, 2]

- K_{love} = [1, 1, -1]
- K_{pizza} = [2, 3, 5]

✅ Value vectors:

- V_{I} = [1, 1, 2]
- V_{love} = [1, 1, -1]
- V_{pizza} = [2, 3, 5]

❓ 1.4 Compute the **raw attention score** for each of the tokens, and fill the table below.

note: $raw\ attention\ score_{i,j} = Q_i \cdot K_j^T$

✅ Answer:

$score_{i,j}$	I	$love$	$pizza$
I	6	0	15
$love$	0	3	0
$pizza$	15	0	38

❓ 1.5 Transform the raw attention score of $love$ into normalized attention scores A_{token} using the Softmax function. Round your results to two decimal places.

✅ Normalized attention score:

A_{love} = [0.05, 0.91, 0.05]

❓ 1.6 Determine the context (weighted sum, or output shown in the graphic) of "love", Z_{love}.

Note: $Z_i = \sum_j A_{i,j} \cdot V_j$

✅ Outputs

- Z_{love} = [1.06, 1.11, -0.56]

❓ 1.7 How does the attention mechanism allow the model to focus on relevant tokens in the input sequence?

✅ Answer: The attention mechanism allows the model to compare each token with the other tokens in the sequence and assign a weight to each one based on how relevant it is. Tokens that are more important or more related receive higher attention scores, so they contribute more to the final representation of the token.

Section 2: Variational AutoEncoder (VAE) For Image Data

In this section, we will build a variational autoencoder (VAE) model using the MNIST dataset and generate new data points.

2.1 Data preparation

Let's begin by importing some libraries.

In [1]:
```
# loading libraries
!pip install torchvision
import torch
import numpy as np
import torch.nn as nn
import matplotlib.pyplot as plt
from torchvision.datasets import MNIST
from torch.utils.data import DataLoader
import torchvision.transforms as transforms
from mpl_toolkits.axes_grid1 import ImageGrid
from torchvision.utils import save_image, make_grid
```

```
Requirement already satisfied: torchvision in /Users/dianadayekh/anaconda3/e
nvs/clean_cluster/lib/python3.11/site-packages (0.26.0)
Requirement already satisfied: numpy in /Users/dianadayekh/anaconda3/envs/cl
ean_cluster/lib/python3.11/site-packages (from torchvision) (1.26.4)
Requirement already satisfied: torch==2.11.0 in /Users/dianadayekh/anaconda
3/envs/clean_cluster/lib/python3.11/site-packages (from torchvision) (2.11.
0)
Requirement already satisfied: pillow!=8.3.*,>=5.3.0 in /Users/dianadayekh/a
naconda3/envs/clean_cluster/lib/python3.11/site-packages (from torchvision)
(12.1.1)
Requirement already satisfied: filelock in /Users/dianadayekh/anaconda3/env
s/clean_cluster/lib/python3.11/site-packages (from torch==2.11.0->torchvisio
n) (3.25.2)
Requirement already satisfied: typing-extensions>=4.10.0 in /Users/dianadaye
kh/anaconda3/envs/clean_cluster/lib/python3.11/site-packages (from torch==2.
11.0->torchvision) (4.15.0)
Requirement already satisfied: setuptools<82 in /Users/dianadayekh/anaconda
3/envs/clean_cluster/lib/python3.11/site-packages (from torch==2.11.0->torch
vision) (80.10.2)
Requirement already satisfied: sympy>=1.13.3 in /Users/dianadayekh/anaconda
3/envs/clean_cluster/lib/python3.11/site-packages (from torch==2.11.0->torch
vision) (1.14.0)
Requirement already satisfied: networkx>=2.5.1 in /Users/dianadayekh/anacond
a3/envs/clean_cluster/lib/python3.11/site-packages (from torch==2.11.0->torc
hvision) (3.6.1)
Requirement already satisfied: jinja2 in /Users/dianadayekh/anaconda3/envs/c
lean_cluster/lib/python3.11/site-packages (from torch==2.11.0->torchvision)
(3.1.6)
Requirement already satisfied: fsspec>=0.8.5 in /Users/dianadayekh/anaconda
3/envs/clean_cluster/lib/python3.11/site-packages (from torch==2.11.0->torch
vision) (2026.3.0)
Requirement already satisfied: mpmath<1.4,>=1.1.0 in /Users/dianadayekh/anac
onda3/envs/clean_cluster/lib/python3.11/site-packages (from sympy>=1.13.3->t
orch==2.11.0->torchvision) (1.3.0)
Requirement already satisfied: MarkupSafe>=2.0 in /Users/dianadayekh/anacond
a3/envs/clean_cluster/lib/python3.11/site-packages (from jinja2->torch==2.1
1.0->torchvision) (3.0.2)
```

In [2]:
```
# create a transform to apply to each datapoint
transform = transforms.Compose([transforms.ToTensor()])
```

❓ 2.1.1 What does the transform do?

✅ Answer:

- turns the image into a PyTorch tensor, which is just the type of object PyTorch works with
- changes the pixel numbers to a smaller scale, from 0–255 to 0–1

❓ 2.1.2 Load and preprocess the MNIST dataset.

- load the dataset using MNIST
- create dataloaders using DataLoader

- be sure to set `shuffle=False` to make sure the first image you display here is always the first image later in your code.

In [3]:
```
# load the MNIST datasets and preprocess the data using the transform you ju
path = './data'
data = MNIST(root=path, train=True, download=True, transform=transform)

# create train and test dataloaders
batch_size = 100
data_loader = DataLoader(data, batch_size=batch_size, shuffle=False) #access
```

Run the code below to make sure you are utilizing GPU resources.

In [4]:
```
# select the appropriate hardware
device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
print(device)
```

```
cpu
```

2.2 Check the data

In [5]:
```
# build iterators to simplify data retrieval
dataiter = iter(data_loader)
image = next(dataiter)
print(image[0].shape)
```

```
torch.Size([100, 1, 28, 28])
```

❓ 2.2.1 Explain the shape shown above.

✅ Answers:

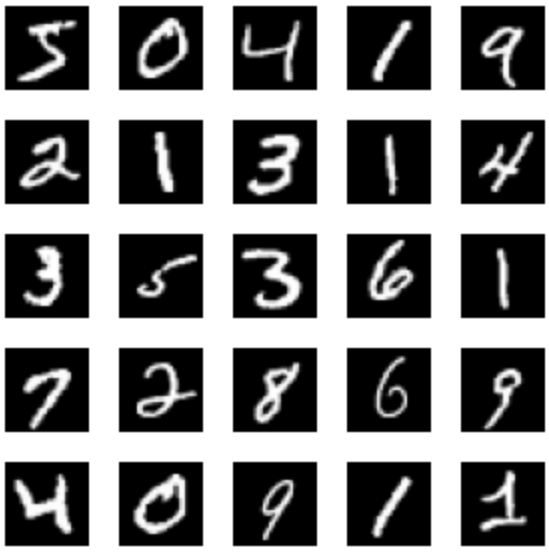
- 100 = there are 100 images in this batch
- 1 = each image has 1 channel, because MNIST is grayscale
- 28 = image height is 28 pixels
- 28 = image width is 28 pixels

❓ 2.2.2 Visualize the first 25 images in a 5×5 grid.

- Set the overall figure size to (3,3).
- Hide image axis to simplify the visualization.
- Use "gray" color map.

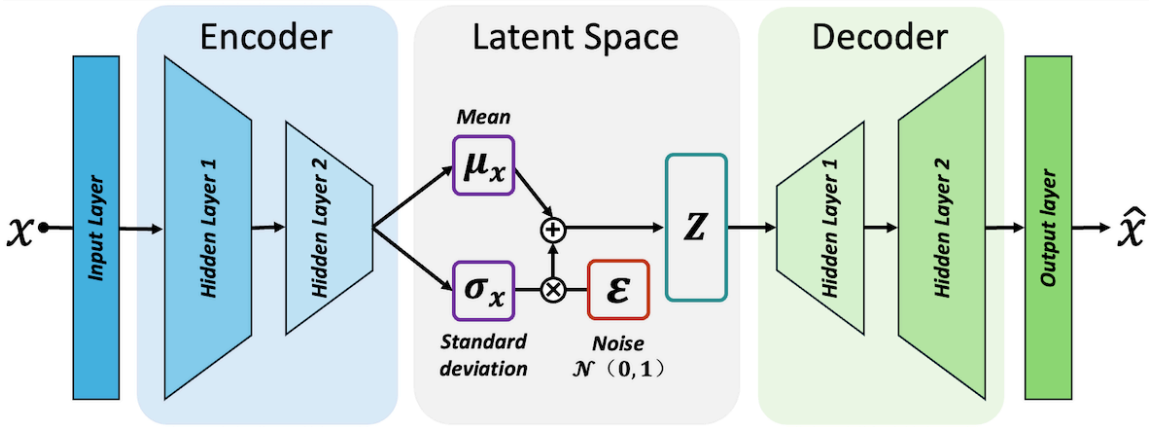
In [6]:
```
# write your code below
fig, axes = plt.subplots(5, 5, figsize=(3, 3))

for i, ax in enumerate(axes.flat):
    ax.imshow(image[0][i].squeeze(), cmap="gray")
    ax.axis("off")
```

```
plt.tight_layout()
plt.show()
```



2.3 Build a VAE

❓ 2.3.1 Now, use the structure and instructions below to complete the construction of the VAE model.



- Both the encoder and the decoder are fully connected. (Linear)
- Both hidden layers in the encoder use Leaky ReLU function to introduce non-linearity. Set the slope of the negative part to 0.2.
- The structure of the decoder is exactly mirrored to that of the encoder, with the exception of an additional Sigmoid function to normalize the output.

In [13]:
```
class VAE(nn.Module):
    def __init__(self, input_dim=784, hidden_dim=400, latent_dim=200, device
        super(VAE, self).__init__()
```

```
        # encoder - complete the code below
        self.encoder = nn.Sequential(
            nn.Linear(input_dim, hidden_dim),
            nn.LeakyReLU(0.2),
            nn.Linear(hidden_dim, latent_dim),
            nn.LeakyReLU(0.2)
        )

        # latent mean and log(variance)
        self.mean_layer = nn.Linear(latent_dim, 2)
        self.logvar_layer = nn.Linear(latent_dim, 2)

        # decoder - complete the code below
        self.decoder = nn.Sequential (
        nn.Linear(2, latent_dim),
        nn.LeakyReLU(0.2),
        nn.Linear(latent_dim, hidden_dim),
        nn.LeakyReLU(0.2),
        nn.Linear(hidden_dim, input_dim),
        nn.Sigmoid() )

    # encoding method
    def encode(self, x):
        x = self.encoder(x)
        mean, logvar = self.mean_layer(x), self.logvar_layer(x)
        return mean, logvar

    # reparameterization trick
    def reparameterization(self, mean, logvar):
        # Convert log(variance) to standard deviation
        std = torch.exp(0.5 * logvar)
        epsilon = torch.randn_like(std).to(device)
        z = mean + std * epsilon
        return z

    # decoding method
    def decode(self, x):
        return self.decoder(x)

    # forward pass
    def forward(self, x):
        mean, logvar = self.encode(x)
        z = self.reparameterization(mean, logvar)
        x_hat = self.decode(z)
        return x_hat, mean, logvar
```

Define our model and use Adam as the optimizer.

In [14]:
```
# define the model
from torch.optim import Adam
model = VAE().to(device)
optimizer = Adam(model.parameters(), lr=1e-3)
```

❓ 2.3.2 Combine the reconstruction loss and the Kullback-Leibler divergence (KLD) to form the loss function.

- The reproduction loss can be represented by binary cross entropy. Sum up all losses.
- $KLD = -0.5 \times \sum(1 + log(variance) - u^2 - variance)$

In [23]:
```
# define the loss function
def loss_function(x, x_hat, mean, logvar):
    bce = nn.BCELoss(reduction='sum')
    recon_loss = bce(x_hat, x)
    kld = -0.5 * torch.sum(1 + logvar - mean.pow(2) - logvar.exp())
    return recon_loss + kld

# reconstruction loss: how close is the reconstructed image to the original
# KL divergence loss: measures wether the latent space is a smooth normal di
```

Finally, we can train our model.

❓ 2.3.3 There is one place you need to write a bit of code below.

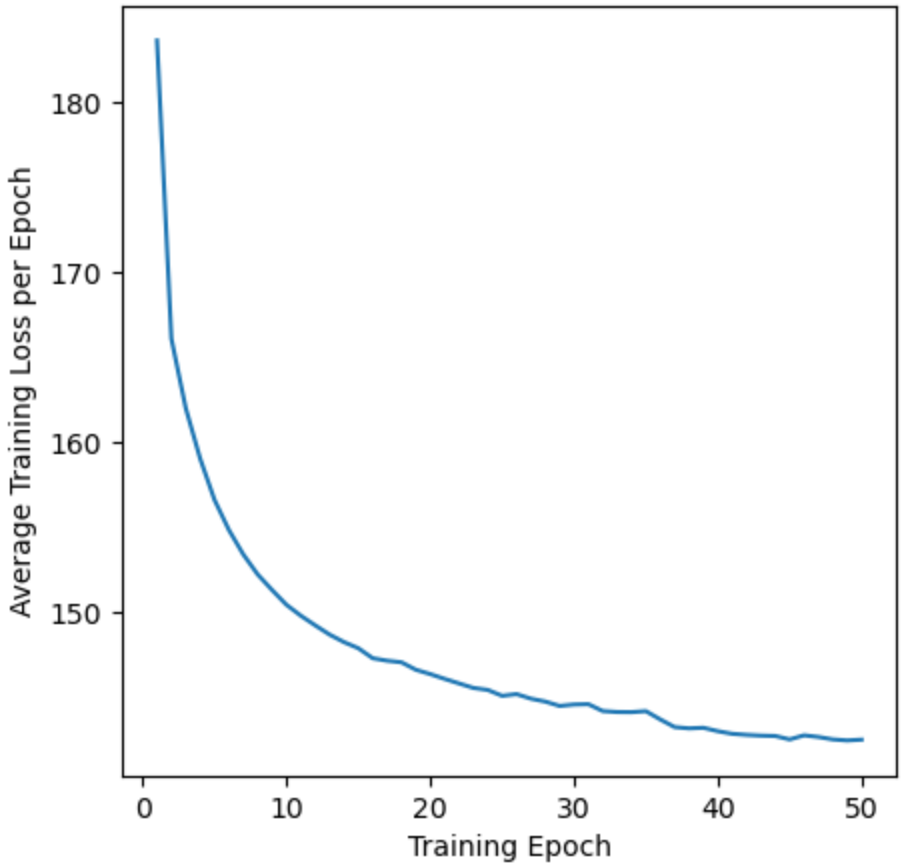
In [24]:
```
import sys
!{sys.executable} -m pip install tqdm
from tqdm import tqdm

def train(model, optimizer, epochs, device, x_dim=784):
    # Set the model to training mode
    model.train()

    # Keep track of training loss
    loss_avg_all = []

    # Loop over the specified number of epochs
    for epoch in tqdm(range(epochs), desc='training...'):
        overall_loss = 0  # Initialize the total loss for the epoch

        # Iterate over each batch from the training data
        for batch_idx, (x, _) in enumerate(data_loader):
            # Flatten the input images and move them to the specified device
            x = x.view(batch_size, x_dim).to(device)

            # Reset the gradients from the previous step
            optimizer.zero_grad()

            # complete the line below
            x_hat, mean, logvar = model(x)

            # Calculate the loss using the reconstruction and KLD loss compo
            loss = loss_function(x, x_hat, mean, logvar)

            # Accumulate the total loss for this epoch
            overall_loss += loss.item()

            # Backpropagation: compute gradients with respect to the loss
            loss.backward()
```

```
            # Update the model parameters using the optimizer
            optimizer.step()

        # Calculate the average loss for the epoch
        loss_avg = overall_loss / (batch_idx * batch_size)
        loss_avg_all.append(loss_avg)


    # Display the training loss
    fig = plt.figure(figsize=(5, 5))
    plt.plot(np.arange(1, epochs + 1), loss_avg_all)
    plt.xlabel('Training Epoch')
    plt.ylabel('Average Training Loss per Epoch')
    plt.show()
```

```
Requirement already satisfied: tqdm in /Users/dianadayekh/anaconda3/envs/cle
an_cluster/lib/python3.11/site-packages (4.67.3)
```

In [25]:
```
# Train the model for 50 epochs and save the distribution of the final later
train(model, optimizer, epochs=50, device=device)
```

```
training...: 100%|██████████████████████████████| 50/50 [03:09<00:00,  3.80
s/it]
```



Once the model is trained, we need to retrieve the learned latent space.

In [26]:
```
def get_final_distributions(model, data_loader, device, x_dim=784):
    model.eval()  # Set the model to evaluation mode
    final_means = []
    final_logvars = []
```

```
    with torch.no_grad():  # Disable gradient computation for inference
        for x, _ in data_loader:
            x = x.view(-1, x_dim).to(device)  # Flatten input and move to de
            _, mean, logvar = model(x)
            final_means.append(mean.cpu())
            final_logvars.append(logvar.cpu())

    # Concatenate all batches to get the full distribution
    final_means = torch.cat(final_means, dim=0)
    final_logvars = torch.cat(final_logvars, dim=0)

    return final_means, final_logvars
```

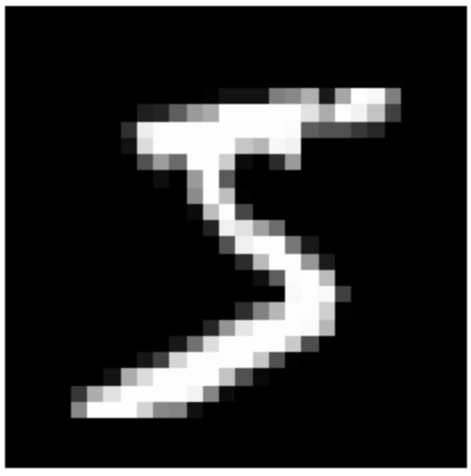
In [27]:
```
# After training, get the final latent distributions
means_final, logvars_final = get_final_distributions(model, data_loader, dev
```

2.4 Generate new images

❓ 2.4.1 First let's visualize the first image in the dataset.

- Set the overall figure size to (3,3).
- Hide image axis to simplify the visualization.
- Use "gray" color map.

In [28]:
```
# write your code below
fig = plt.figure(figsize=(3, 3))
plt.imshow(data[0][0].squeeze(), cmap="gray")
plt.axis("off")
plt.show()
```



Now let's generate some new images based on this image.

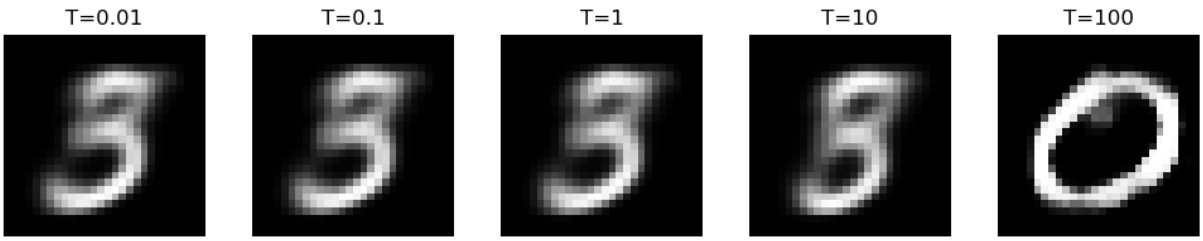
In [29]:
```
# Define a function to generate new images
def generate_digit(mean, logvar, temperature=1.0, device=device):
    # Set the model to evaluation mode
    model.eval()
```

```
    # Check if mean and logvar are already tensors, if not convert them
    if not torch.is_tensor(mean):
        mean = torch.tensor(mean, dtype=torch.float).to(device)
    else:
        mean = mean.to(device)

    if not torch.is_tensor(logvar):
        logvar = torch.tensor(logvar, dtype=torch.float).to(device)
    else:
        logvar = logvar.to(device)

    # Convert log variance to standard deviation
    std = torch.exp(0.5 * logvar)

    # Apply the temperature factor to control diversity
    std = std * temperature

    # Sample noise from a standard normal distribution
    epsilon = torch.randn_like(std).to(device)

    # Use the reparameterization trick to sample z
    z_sample = mean + std * epsilon

    # Decode the latent vector z_sample into the reconstructed digit using t
    with torch.no_grad():  # Disable gradient calculation for faster inferen
        x_decoded = model.decode(z_sample)

    # We reshape because the decoder outputs a flattened vector, and we need
    digit = x_decoded.detach().cpu().reshape(28, 28)

    return digit
```

❓ 2.4.2 Generate 5 new images based on just the first image in the dataset (which you displayed in 2.4.1 above), and test the effect of different "temperature" values from the set [0.01, 0.1, 1, 10, 100]. You may run your code a couple of times to see the difference.

In [30]:
```
# write your code below
temps = [0.01, 0.1, 1, 10, 100]

# first image in the dataset
first_img = data[0][0].view(1, -1).to(device)

# get its latent mean and logvar
model.eval()
with torch.no_grad():
    mean, logvar = model.encode(first_img)

# generate 5 new images with different temperatures
fig, axes = plt.subplots(1, 5, figsize=(10, 2))

for i, temp in enumerate(temps):
    new_digit = generate_digit(mean, logvar, temperature=temp, device=device
    axes[i].imshow(new_digit, cmap="gray")
    axes[i].axis("off")
```

```
    axes[i].set_title(f"T={temp}")

plt.tight_layout()
plt.show()
```



❓ 2.4.3 Describe your observations and explain the role of the temperature parameter in the context of this VAE. *Consider how the* `temperature` *parameter is used in the* `generate_digit` *function. Compare your generated images to the first image in the dataset that you displayed in 2.4.1.*

✅ Answer: The lower temperature values produced images that looked more like the original first image, while higher temperature values created more variation and distortion. This happens because the temperature scales the standard deviation in the latent space before sampling. Lower temperatures add less noise, so the output stays close to the original, while higher temperatures add more noise, making the generated images more diverse but less clear.

Section 3: SeqToSeq For Molecules

The Seq2Seq architecture also consists of an encoder and decoder, and can be trained to function as a generative model.

Let's see how this model performs on molecular data!

3.1 Data preparation

❓ 3.1.1 Please follow the instructions below:

- Load the given data set **alcohol_mol.csv** as a dataframe and save all smiles into **smiles**.
- Save the first 10 SMILES strings as a list called **train_smiles**, and store the remaining SMILES strings as **test_smiles**.
- Determine the max length of all smiles **max_length**, and save unique tokens of all smiles into the list variable **tokens**.

In [36]:
```
# write your code below
import pandas as pd
df = pd.read_csv("alcohol_mol.csv")
print(df.columns)
smiles = df["SMILE"].tolist()
train_smiles = smiles[:10]
test_smiles = smiles[10:]
max_length = max(len(s) for s in smiles)
tokens = sorted(set("".join(smiles)))
```

```
Index(['SMILE'], dtype='str')
['CCCCCCCCCO', 'CCCCCCCCCCCCCCCCCO', 'CCCCCCCCCCCCCCCCO', 'CCCCCO', 'CCCCCC
CCCCCO', 'CCCCCCO', 'CCCCCCCCO', 'CCCCCCCCCCCCCCO', 'CCCCCCCCCCO', 'CCCC
O', 'CCO', 'CO', 'CCCCCCO', 'CCCCCCCCCCCCCO', 'CCCCCCCCCO', 'CCCO', 'CCCCC
CCCCCCCCCO', 'CCCCCCCCCCCCCCCCCO']
```

❓ 3.1.2 Briefly qualitatively describe the molecular structures we're working with in this dataset.

In [37]:
```
# feel free to know your data using any methods you've learned so far
df.head(10)
```

Out[37]:

	SMILE
0	CCCCCCCCCO
1	CCCCCCCCCCCCCCCCCCO
2	CCCCCCCCCCCCCCCCO
3	CCCCCO
4	CCCCCCCCCCCCO
5	CCCCCCO
6	CCCCCCCCO
7	CCCCCCCCCCCCCCO
8	CCCCCCCCCCO
9	CCCCO

✅ Answers: Our data are different molecular structures represented as SMILES strings, where each molecule is encoded using a sequence of characters.

3.2 Model preparation (only 1 question needs to be answered in this sub-section)

The following code may take approximately 2 minutes to complete.

You will likely see some warnings, ignore these.

Run these in google colab with python 3.12.

When you see a "Restart Session Warning" pop-up, click "***Cancel***".

This warning appears due to library incompatibilities with preinstalled packages in Google Colab.

In [38]:
```
# remove incompatible tf and keras
!pip uninstall -y keras tensorflow tensorflow-probability
```

```
Found existing installation: keras 3.13.2
Uninstalling keras-3.13.2:
  Successfully uninstalled keras-3.13.2
Found existing installation: tensorflow 2.21.0
Uninstalling tensorflow-2.21.0:
  Successfully uninstalled tensorflow-2.21.0
WARNING: Skipping tensorflow-probability as it is not installed.
```

In [39]:
```
# install compatible versions of tensorflow and keras
!pip install tensorflow==2.15.0
!pip install keras==2.15.0
!pip install tensorflow-probability==0.23.0
```

```
Collecting tensorflow==2.15.0
  Downloading tensorflow-2.15.0-cp311-cp311-macosx_12_0_arm64.whl.metadata
(3.6 kB)
Collecting tensorflow-macos==2.15.0 (from tensorflow==2.15.0)
  Downloading tensorflow_macos-2.15.0-cp311-cp311-macosx_12_0_arm64.whl.meta
data (4.2 kB)
Requirement already satisfied: absl-py>=1.0.0 in /Users/dianadayekh/anaconda
3/envs/clean_cluster/lib/python3.11/site-packages (from tensorflow-macos==2.
15.0->tensorflow==2.15.0) (2.4.0)
Requirement already satisfied: astunparse>=1.6.0 in /Users/dianadayekh/anaco
nda3/envs/clean_cluster/lib/python3.11/site-packages (from tensorflow-macos=
=2.15.0->tensorflow==2.15.0) (1.6.3)
Requirement already satisfied: flatbuffers>=23.5.26 in /Users/dianadayekh/an
aconda3/envs/clean_cluster/lib/python3.11/site-packages (from tensorflow-mac
os==2.15.0->tensorflow==2.15.0) (25.12.19)
Requirement already satisfied: gast!=0.5.0,!=0.5.1,!=0.5.2,>=0.2.1 in /User
s/dianadayekh/anaconda3/envs/clean_cluster/lib/python3.11/site-packages (fro
m tensorflow-macos==2.15.0->tensorflow==2.15.0) (0.7.0)
Requirement already satisfied: google-pasta>=0.1.1 in /Users/dianadayekh/ana
conda3/envs/clean_cluster/lib/python3.11/site-packages (from tensorflow-maco
s==2.15.0->tensorflow==2.15.0) (0.2.0)
Requirement already satisfied: h5py>=2.9.0 in /Users/dianadayekh/anaconda3/e
nvs/clean_cluster/lib/python3.11/site-packages (from tensorflow-macos==2.15.
0->tensorflow==2.15.0) (3.14.0)
Requirement already satisfied: libclang>=13.0.0 in /Users/dianadayekh/anacon
da3/envs/clean_cluster/lib/python3.11/site-packages (from tensorflow-macos==
2.15.0->tensorflow==2.15.0) (18.1.1)
Collecting ml-dtypes~=0.2.0 (from tensorflow-macos==2.15.0->tensorflow==2.1
5.0)
  Downloading ml_dtypes-0.2.0-cp311-cp311-macosx_10_9_universal2.whl.metadat
a (20 kB)
Requirement already satisfied: numpy<2.0.0,>=1.23.5 in /Users/dianadayekh/an
aconda3/envs/clean_cluster/lib/python3.11/site-packages (from tensorflow-mac
os==2.15.0->tensorflow==2.15.0) (1.26.4)
Requirement already satisfied: opt-einsum>=2.3.2 in /Users/dianadayekh/anaco
nda3/envs/clean_cluster/lib/python3.11/site-packages (from tensorflow-macos=
=2.15.0->tensorflow==2.15.0) (3.4.0)
Requirement already satisfied: packaging in /Users/dianadayekh/anaconda3/env
s/clean_cluster/lib/python3.11/site-packages (from tensorflow-macos==2.15.0-
>tensorflow==2.15.0) (25.0)
Collecting protobuf!=4.21.0,!=4.21.1,!=4.21.2,!=4.21.3,!=4.21.4,!=4.21.5,<5.
0.0dev,>=3.20.3 (from tensorflow-macos==2.15.0->tensorflow==2.15.0)
  Downloading protobuf-4.25.9-cp37-abi3-macosx_10_9_universal2.whl.metadata
(541 bytes)
Requirement already satisfied: setuptools in /Users/dianadayekh/anaconda3/en
vs/clean_cluster/lib/python3.11/site-packages (from tensorflow-macos==2.15.0
->tensorflow==2.15.0) (80.10.2)
Requirement already satisfied: six>=1.12.0 in /Users/dianadayekh/anaconda3/e
nvs/clean_cluster/lib/python3.11/site-packages (from tensorflow-macos==2.15.
0->tensorflow==2.15.0) (1.17.0)
Requirement already satisfied: termcolor>=1.1.0 in /Users/dianadayekh/anacon
da3/envs/clean_cluster/lib/python3.11/site-packages (from tensorflow-macos==
2.15.0->tensorflow==2.15.0) (3.3.0)
Requirement already satisfied: typing-extensions>=3.6.6 in /Users/dianadayek
h/anaconda3/envs/clean_cluster/lib/python3.11/site-packages (from tensorflow
-macos==2.15.0->tensorflow==2.15.0) (4.15.0)
```

```
Collecting wrapt<1.15,>=1.11.0 (from tensorflow-macos==2.15.0->tensorflow==
2.15.0)
  Downloading wrapt-1.14.2-cp311-cp311-macosx_11_0_arm64.whl.metadata (6.5 k
B)
Collecting tensorflow-io-gcs-filesystem>=0.23.1 (from tensorflow-macos==2.1
5.0->tensorflow==2.15.0)
  Downloading tensorflow_io_gcs_filesystem-0.37.1-cp311-cp311-macosx_12_0_ar
m64.whl.metadata (14 kB)
Requirement already satisfied: grpcio<2.0,>=1.24.3 in /Users/dianadayekh/ana
conda3/envs/clean_cluster/lib/python3.11/site-packages (from tensorflow-maco
s==2.15.0->tensorflow==2.15.0) (1.78.0)
Collecting tensorboard<2.16,>=2.15 (from tensorflow-macos==2.15.0->tensorflo
w==2.15.0)
  Downloading tensorboard-2.15.2-py3-none-any.whl.metadata (1.7 kB)
Collecting tensorflow-estimator<2.16,>=2.15.0 (from tensorflow-macos==2.15.0
->tensorflow==2.15.0)
  Downloading tensorflow_estimator-2.15.0-py2.py3-none-any.whl.metadata (1.3
kB)
Collecting keras<2.16,>=2.15.0 (from tensorflow-macos==2.15.0->tensorflow==
2.15.0)
  Downloading keras-2.15.0-py3-none-any.whl.metadata (2.4 kB)
Collecting google-auth<3,>=1.6.3 (from tensorboard<2.16,>=2.15->tensorflow-m
acos==2.15.0->tensorflow==2.15.0)
  Downloading google_auth-2.49.1-py3-none-any.whl.metadata (6.2 kB)
Collecting google-auth-oauthlib<2,>=0.5 (from tensorboard<2.16,>=2.15->tenso
rflow-macos==2.15.0->tensorflow==2.15.0)
  Downloading google_auth_oauthlib-1.3.1-py3-none-any.whl.metadata (2.6 kB)
Collecting markdown>=2.6.8 (from tensorboard<2.16,>=2.15->tensorflow-macos==
2.15.0->tensorflow==2.15.0)
  Downloading markdown-3.10.2-py3-none-any.whl.metadata (5.1 kB)
Requirement already satisfied: requests<3,>=2.21.0 in /Users/dianadayekh/ana
conda3/envs/clean_cluster/lib/python3.11/site-packages (from tensorboard<2.1
6,>=2.15->tensorflow-macos==2.15.0->tensorflow==2.15.0) (2.32.5)
Collecting tensorboard-data-server<0.8.0,>=0.7.0 (from tensorboard<2.16,>=2.
15->tensorflow-macos==2.15.0->tensorflow==2.15.0)
  Downloading tensorboard_data_server-0.7.2-py3-none-any.whl.metadata (1.1 k
B)
Collecting werkzeug>=1.0.1 (from tensorboard<2.16,>=2.15->tensorflow-macos==
2.15.0->tensorflow==2.15.0)
  Downloading werkzeug-3.1.8-py3-none-any.whl.metadata (4.0 kB)
Collecting pyasn1-modules>=0.2.1 (from google-auth<3,>=1.6.3->tensorboard<2.
16,>=2.15->tensorflow-macos==2.15.0->tensorflow==2.15.0)
  Downloading pyasn1_modules-0.4.2-py3-none-any.whl.metadata (3.5 kB)
Collecting cryptography>=38.0.3 (from google-auth<3,>=1.6.3->tensorboard<2.1
6,>=2.15->tensorflow-macos==2.15.0->tensorflow==2.15.0)
  Downloading cryptography-46.0.7-cp311-abi3-macosx_10_9_universal2.whl.meta
data (5.7 kB)
Collecting requests-oauthlib>=0.7.0 (from google-auth-oauthlib<2,>=0.5->tens
orboard<2.16,>=2.15->tensorflow-macos==2.15.0->tensorflow==2.15.0)
  Downloading requests_oauthlib-2.0.0-py2.py3-none-any.whl.metadata (11 kB)
Requirement already satisfied: charset_normalizer<4,>=2 in /Users/dianadayek
h/anaconda3/envs/clean_cluster/lib/python3.11/site-packages (from requests<
3,>=2.21.0->tensorboard<2.16,>=2.15->tensorflow-macos==2.15.0->tensorflow==
2.15.0) (3.4.4)
Requirement already satisfied: idna<4,>=2.5 in /Users/dianadayekh/anaconda3/
envs/clean_cluster/lib/python3.11/site-packages (from requests<3,>=2.21.0->t
```

```
ensorboard<2.16,>=2.15->tensorflow-macos==2.15.0->tensorflow==2.15.0) (3.11)
Requirement already satisfied: urllib3<3,>=1.21.1 in /Users/dianadayekh/anac
onda3/envs/clean_cluster/lib/python3.11/site-packages (from requests<3,>=2.2
1.0->tensorboard<2.16,>=2.15->tensorflow-macos==2.15.0->tensorflow==2.15.0)
(2.6.3)
Requirement already satisfied: certifi>=2017.4.17 in /Users/dianadayekh/anac
onda3/envs/clean_cluster/lib/python3.11/site-packages (from requests<3,>=2.2
1.0->tensorboard<2.16,>=2.15->tensorflow-macos==2.15.0->tensorflow==2.15.0)
(2026.1.4)
Requirement already satisfied: wheel<1.0,>=0.23.0 in /Users/dianadayekh/anac
onda3/envs/clean_cluster/lib/python3.11/site-packages (from astunparse>=1.6.
0->tensorflow-macos==2.15.0->tensorflow==2.15.0) (0.46.3)
Requirement already satisfied: cffi>=2.0.0 in /Users/dianadayekh/anaconda3/e
nvs/clean_cluster/lib/python3.11/site-packages (from cryptography>=38.0.3->g
oogle-auth<3,>=1.6.3->tensorboard<2.16,>=2.15->tensorflow-macos==2.15.0->ten
sorflow==2.15.0) (2.0.0)
Requirement already satisfied: pycparser in /Users/dianadayekh/anaconda3/env
s/clean_cluster/lib/python3.11/site-packages (from cffi>=2.0.0->cryptography
>=38.0.3->google-auth<3,>=1.6.3->tensorboard<2.16,>=2.15->tensorflow-macos==
2.15.0->tensorflow==2.15.0) (2.23)
Collecting pyasn1<0.7.0,>=0.6.1 (from pyasn1-modules>=0.2.1->google-auth<3,>
=1.6.3->tensorboard<2.16,>=2.15->tensorflow-macos==2.15.0->tensorflow==2.15.
0)
  Downloading pyasn1-0.6.3-py3-none-any.whl.metadata (8.4 kB)
Collecting oauthlib>=3.0.0 (from requests-oauthlib>=0.7.0->google-auth-oauth
lib<2,>=0.5->tensorboard<2.16,>=2.15->tensorflow-macos==2.15.0->tensorflow==
2.15.0)
  Downloading oauthlib-3.3.1-py3-none-any.whl.metadata (7.9 kB)
Requirement already satisfied: markupsafe>=2.1.1 in /Users/dianadayekh/anaco
nda3/envs/clean_cluster/lib/python3.11/site-packages (from werkzeug>=1.0.1->
tensorboard<2.16,>=2.15->tensorflow-macos==2.15.0->tensorflow==2.15.0) (3.0.
2)
Downloading tensorflow-2.15.0-cp311-cp311-macosx_12_0_arm64.whl (2.1 kB)
Downloading tensorflow_macos-2.15.0-cp311-cp311-macosx_12_0_arm64.whl (208.8
MB)
   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 208.8/208.8 MB 20.9 MB/s  0:00:1
0m0:00:0100:01
Downloading keras-2.15.0-py3-none-any.whl (1.7 MB)
   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 1.7/1.7 MB 23.0 MB/s  0:00:00
Downloading ml_dtypes-0.2.0-cp311-cp311-macosx_10_9_universal2.whl (1.2 MB)
   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 1.2/1.2 MB 27.5 MB/s  0:00:00
Downloading protobuf-4.25.9-cp37-abi3-macosx_10_9_universal2.whl (394 kB)
Downloading tensorboard-2.15.2-py3-none-any.whl (5.5 MB)
   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 5.5/5.5 MB 27.1 MB/s  0:00:00
Downloading google_auth-2.49.1-py3-none-any.whl (240 kB)
Downloading google_auth_oauthlib-1.3.1-py3-none-any.whl (19 kB)
Downloading tensorboard_data_server-0.7.2-py3-none-any.whl (2.4 kB)
Downloading tensorflow_estimator-2.15.0-py2.py3-none-any.whl (441 kB)
Downloading wrapt-1.14.2-cp311-cp311-macosx_11_0_arm64.whl (35 kB)
Downloading cryptography-46.0.7-cp311-abi3-macosx_10_9_universal2.whl (7.2 M
B)
   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 7.2/7.2 MB 13.2 MB/s  0:00:00eta
0:00:01m
Downloading markdown-3.10.2-py3-none-any.whl (108 kB)
Downloading pyasn1_modules-0.4.2-py3-none-any.whl (181 kB)
Downloading pyasn1-0.6.3-py3-none-any.whl (83 kB)
```

```
Downloading requests_oauthlib-2.0.0-py2.py3-none-any.whl (24 kB)
Downloading oauthlib-3.3.1-py3-none-any.whl (160 kB)
Downloading tensorflow_io_gcs_filesystem-0.37.1-cp311-cp311-macosx_12_0_arm6
4.whl (3.5 MB)
   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 3.5/3.5 MB 32.7 MB/s  0:00:00
Downloading werkzeug-3.1.8-py3-none-any.whl (226 kB)
Installing collected packages: wrapt, werkzeug, tensorflow-io-gcs-filesyste
m, tensorflow-estimator, tensorboard-data-server, pyasn1, protobuf, oauthli
b, ml-dtypes, markdown, keras, requests-oauthlib, pyasn1-modules, cryptograp
hy, google-auth, google-auth-oauthlib, tensorboard, tensorflow-macos, tensor
flow
  Attempting uninstall: wrapt
    Found existing installation: wrapt 2.1.2
    Uninstalling wrapt-2.1.2:
      Successfully uninstalled wrapt-2.1.2
  Attempting uninstall: protobufm━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━  3/19 [te
nsorflow-estimator]esystem]
    Found existing installation: protobuf 7.34.1━━━━━━━━━━━━━━━━━  3/19 [tensor
flow-estimator]
    Uninstalling protobuf-7.34.1:━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━  3/19 [tensor
flow-estimator]
      Successfully uninstalled protobuf-7.34.1━━━━━━━━━━━━━━━━━  3/19 [tensor
flow-estimator]
  Attempting uninstall: ml-dtypes[0m━━━━━━━━━━━━━━━━━━━━━━━━━━━━  7/19 [oauthli
b]timator]
    Found existing installation: ml_dtypes 0.5.4━━━━━━━━━━━━━━━━  7/19 [oauthl
ib]
    Uninstalling ml_dtypes-0.5.4:[90m━━━━━━━━━━━━━━━━━━━━━━━━━━━  7/19 [oauthl
ib]
      Successfully uninstalled ml_dtypes-0.5.4━━━━━━━━━━━━━━━━━  7/19 [oauthl
ib]
   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 19/19 [tensorflow]9 [tensorflow-
macos]
Successfully installed cryptography-46.0.7 google-auth-2.49.1 google-auth-oa
uthlib-1.3.1 keras-2.15.0 markdown-3.10.2 ml-dtypes-0.2.0 oauthlib-3.3.1 pro
tobuf-4.25.9 pyasn1-0.6.3 pyasn1-modules-0.4.2 requests-oauthlib-2.0.0 tenso
rboard-2.15.2 tensorboard-data-server-0.7.2 tensorflow-2.15.0 tensorflow-est
imator-2.15.0 tensorflow-io-gcs-filesystem-0.37.1 tensorflow-macos-2.15.0 we
rkzeug-3.1.8 wrapt-1.14.2
Requirement already satisfied: keras==2.15.0 in /Users/dianadayekh/anaconda
3/envs/clean_cluster/lib/python3.11/site-packages (2.15.0)
Collecting tensorflow-probability==0.23.0
  Downloading tensorflow_probability-0.23.0-py2.py3-none-any.whl.metadata (1
3 kB)
Requirement already satisfied: absl-py in /Users/dianadayekh/anaconda3/envs/
clean_cluster/lib/python3.11/site-packages (from tensorflow-probability==0.2
3.0) (2.4.0)
Requirement already satisfied: six>=1.10.0 in /Users/dianadayekh/anaconda3/e
nvs/clean_cluster/lib/python3.11/site-packages (from tensorflow-probability=
=0.23.0) (1.17.0)
Requirement already satisfied: numpy>=1.13.3 in /Users/dianadayekh/anaconda
3/envs/clean_cluster/lib/python3.11/site-packages (from tensorflow-probabili
ty==0.23.0) (1.26.4)
Requirement already satisfied: decorator in /Users/dianadayekh/anaconda3/env
s/clean_cluster/lib/python3.11/site-packages (from tensorflow-probability==
0.23.0) (5.2.1)
```

```
Collecting cloudpickle>=1.3 (from tensorflow-probability==0.23.0)
  Downloading cloudpickle-3.1.2-py3-none-any.whl.metadata (7.1 kB)
Requirement already satisfied: gast>=0.3.2 in /Users/dianadayekh/anaconda3/e
nvs/clean_cluster/lib/python3.11/site-packages (from tensorflow-probability=
=0.23.0) (0.7.0)
Collecting dm-tree (from tensorflow-probability==0.23.0)
  Downloading dm_tree-0.1.10-cp311-cp311-macosx_10_9_universal2.whl.metadata
(2.6 kB)
Requirement already satisfied: attrs>=18.2.0 in /Users/dianadayekh/anaconda
3/envs/clean_cluster/lib/python3.11/site-packages (from dm-tree->tensorflow-
probability==0.23.0) (25.4.0)
Requirement already satisfied: wrapt>=1.11.2 in /Users/dianadayekh/anaconda
3/envs/clean_cluster/lib/python3.11/site-packages (from dm-tree->tensorflow-
probability==0.23.0) (1.14.2)
Downloading tensorflow_probability-0.23.0-py2.py3-none-any.whl (6.9 MB)
   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 6.9/6.9 MB 21.7 MB/s  0:00:00 et
a 0:00:01
Downloading cloudpickle-3.1.2-py3-none-any.whl (22 kB)
Downloading dm_tree-0.1.10-cp311-cp311-macosx_10_9_universal2.whl (314 kB)
Installing collected packages: dm-tree, cloudpickle, tensorflow-probability
   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 3/3 [tensorflow-probability]rflo
w-probability]
Successfully installed cloudpickle-3.1.2 dm-tree-0.1.10 tensorflow-probabili
ty-0.23.0
```

In [40]:
```
# install deepchem
!pip install git+https://github.com/deepchem/deepchem.git@master
```

```
Collecting git+https://github.com/deepchem/deepchem.git@master
  Cloning https://github.com/deepchem/deepchem.git (to revision master) to /
private/var/folders/80/xj_4_j2d1hsgpwv5r81vhhc40000gn/T/pip-req-build-r2b43u
c7
  Running command git clone --filter=blob:none --quiet https://github.com/de
epchem/deepchem.git /private/var/folders/80/xj_4_j2d1hsgpwv5r81vhhc40000gn/
T/pip-req-build-r2b43uc7
  Resolved https://github.com/deepchem/deepchem.git to commit 95a512b5c68683
f0c6e2b0f7cad773cb6a7d822c
  Installing build dependencies ... done
  Getting requirements to build wheel ... done
  Preparing metadata (pyproject.toml) ... done
Requirement already satisfied: joblib in /Users/dianadayekh/anaconda3/envs/c
lean_cluster/lib/python3.11/site-packages (from deepchem==2.8.1.dev202604082
00428) (1.5.3)
Requirement already satisfied: numpy<2 in /Users/dianadayekh/anaconda3/envs/
clean_cluster/lib/python3.11/site-packages (from deepchem==2.8.1.dev20260408
200428) (1.26.4)
Requirement already satisfied: pandas in /Users/dianadayekh/anaconda3/envs/c
lean_cluster/lib/python3.11/site-packages (from deepchem==2.8.1.dev202604082
00428) (3.0.1)
Requirement already satisfied: scikit-learn in /Users/dianadayekh/anaconda3/
envs/clean_cluster/lib/python3.11/site-packages (from deepchem==2.8.1.dev202
60408200428) (1.8.0)
Requirement already satisfied: sympy in /Users/dianadayekh/anaconda3/envs/cl
ean_cluster/lib/python3.11/site-packages (from deepchem==2.8.1.dev2026040820
0428) (1.14.0)
Requirement already satisfied: scipy>=1.10.1 in /Users/dianadayekh/anaconda
3/envs/clean_cluster/lib/python3.11/site-packages (from deepchem==2.8.1.dev2
0260408200428) (1.17.1)
Requirement already satisfied: rdkit in /Users/dianadayekh/anaconda3/envs/cl
ean_cluster/lib/python3.11/site-packages (from deepchem==2.8.1.dev2026040820
0428) (2025.9.5)
Requirement already satisfied: python-dateutil>=2.8.2 in /Users/dianadayekh/
anaconda3/envs/clean_cluster/lib/python3.11/site-packages (from pandas->deep
chem==2.8.1.dev20260408200428) (2.9.0.post0)
Requirement already satisfied: six>=1.5 in /Users/dianadayekh/anaconda3/env
s/clean_cluster/lib/python3.11/site-packages (from python-dateutil>=2.8.2->p
andas->deepchem==2.8.1.dev20260408200428) (1.17.0)
Requirement already satisfied: Pillow in /Users/dianadayekh/anaconda3/envs/c
lean_cluster/lib/python3.11/site-packages (from rdkit->deepchem==2.8.1.dev20
260408200428) (12.1.1)
Requirement already satisfied: threadpoolctl>=3.2.0 in /Users/dianadayekh/an
aconda3/envs/clean_cluster/lib/python3.11/site-packages (from scikit-learn->
deepchem==2.8.1.dev20260408200428) (3.5.0)
Requirement already satisfied: mpmath<1.4,>=1.1.0 in /Users/dianadayekh/anac
onda3/envs/clean_cluster/lib/python3.11/site-packages (from sympy->deepchem=
=2.8.1.dev20260408200428) (1.3.0)
Building wheels for collected packages: deepchem
  Building wheel for deepchem (pyproject.toml) ... done
  Created wheel for deepchem: filename=deepchem-2.8.1.dev20260408200428-py3-
none-any.whl size=1340915 sha256=523297666156a3c251284d0721a1bb4de683b0911cf
255e960fc4bfea5b9b70c
  Stored in directory: /private/var/folders/80/xj_4_j2d1hsgpwv5r81vhhc40000g
n/T/pip-ephem-wheel-cache-0lb752z3/wheels/3d/79/5c/19c87add2e6b21082614fa299
3efc61d461b91dab09c2ecb73
```

```
Successfully built deepchem
Installing collected packages: deepchem
Successfully installed deepchem-2.8.1.dev20260408200428
```

In [41]:
```python
# prepare the model
import deepchem as dc
from deepchem.models.optimizers import ExponentialDecay
batch_size = 1
batches_per_epoch = len(train_smiles)/batch_size

model = dc.models.SeqToSeq(
    input_tokens=tokens,
    output_tokens=tokens,
    max_output_length=max_length,
    encoder_layers=2,
    decoder_layers=2,
    embedding_dimension=64,
    model_dir='fingerprint',
    batch_size=batch_size,
    learning_rate=ExponentialDecay(0.001, 0.9, batches_per_epoch)
    )
```

```
WARNING:tensorflow:From /Users/dianadayekh/anaconda3/envs/clean_cluster/lib/
python3.11/site-packages/tensorflow/python/util/deprecation.py:588: calling
function (from tensorflow.python.eager.polymorphic_function.polymorphic_func
tion) with experimental_relax_shapes is deprecated and will be removed in a
future version.
Instructions for updating:
experimental_relax_shapes is deprecated, use reduce_retracing instead
Skipped loading some PyTorch models, missing a dependency. No module named
'transformers'
No module named 'transformers'
Skipped loading modules with pytorch-geometric dependency, missing a depende
ncy. No module named 'transformers'
Skipped loading modules with pytorch-lightning dependency, missing a depende
ncy. No module named 'lightning'
Skipped loading some Jax models, missing a dependency. No module named 'jax'
```

Click here to learn more about DeepChem, a powerful collection of deep learning models for chemistry applications.

3.3 Model training and evaluation

❓ 3.3.1 Please follow the instructions below:

- Define a generator function called **generate_sequences**. This function should control how many times the entire training set **train_smiles** is iterated over, where each full pass through the dataset is defined as one epoch.
- Within each epoch, generate a tuple **(input_smile, output_smile)** for each SMILES string in **train_smiles**.
- It is helpful to display progress updates to monitor the status of your computation.

In [42]:
```python
# define the generator
def generate_sequences(epochs, train_smiles=train_smiles):
    for epoch in range(epochs):
        print(f"Epoch {epoch+1}/{epochs}")
        for s in train_smiles:
            yield (s, s)
```

In [43]:
```python
# model training
model.fit_sequences(generate_sequences(100))
```

```
Epoch 1/100
Epoch 2/100
Epoch 3/100
Epoch 4/100
Epoch 5/100
Epoch 6/100
Epoch 7/100
Epoch 8/100
Epoch 9/100
Epoch 10/100
Epoch 11/100
Epoch 12/100
Epoch 13/100
Epoch 14/100
Epoch 15/100
Epoch 16/100
Epoch 17/100
Epoch 18/100
Epoch 19/100
Epoch 20/100
Epoch 21/100
Epoch 22/100
Epoch 23/100
Epoch 24/100
Epoch 25/100
Epoch 26/100
Epoch 27/100
Epoch 28/100
Epoch 29/100
Epoch 30/100
Epoch 31/100
Epoch 32/100
Epoch 33/100
Epoch 34/100
Epoch 35/100
Epoch 36/100
Epoch 37/100
Epoch 38/100
Epoch 39/100
Epoch 40/100
Epoch 41/100
Epoch 42/100
Epoch 43/100
Epoch 44/100
Epoch 45/100
Epoch 46/100
Epoch 47/100
Epoch 48/100
Epoch 49/100
Epoch 50/100
Epoch 51/100
Epoch 52/100
Epoch 53/100
Epoch 54/100
Epoch 55/100
Epoch 56/100
```

```
Epoch 57/100
Epoch 58/100
Epoch 59/100
Epoch 60/100
Epoch 61/100
Epoch 62/100
Epoch 63/100
Epoch 64/100
Epoch 65/100
Epoch 66/100
Epoch 67/100
Epoch 68/100
Epoch 69/100
Epoch 70/100
Epoch 71/100
Epoch 72/100
Epoch 73/100
Epoch 74/100
Epoch 75/100
Epoch 76/100
Epoch 77/100
Epoch 78/100
Epoch 79/100
Epoch 80/100
Epoch 81/100
Epoch 82/100
Epoch 83/100
Epoch 84/100
Epoch 85/100
Epoch 86/100
Epoch 87/100
Epoch 88/100
Epoch 89/100
Epoch 90/100
Epoch 91/100
Epoch 92/100
Epoch 93/100
Epoch 94/100
Epoch 95/100
Epoch 96/100
Epoch 97/100
Epoch 98/100
Epoch 99/100
Epoch 100/100
```

❓ 3.3.2 Calculate the total number of data points and the accuracy of correct SMILES regeneration for both the training and test datasets.

Useful link: predict_from_sequences

In [50]:
```python
# write your code below
def regeneration_accuracy(smiles_list, name):
    preds = model.predict_from_sequences(smiles_list)

    total = len(smiles_list)
```

```
    correct = 0

    for pred, true in zip(preds, smiles_list):
        pred_str = "".join(pred).strip()
        true_str = str(true).strip()
        if pred_str == true_str:
            correct += 1

    acc = correct / total if total > 0 else 0

    print(f"{name} total data points: {total}")
    print(f"{name} correct regenerations: {correct}")
    print(f"{name} accuracy: {acc:.2%}")
    print()

    return total, correct, acc

train_total, train_correct, train_acc = regeneration_accuracy(train_smiles,
test_total, test_correct, test_acc = regeneration_accuracy(test_smiles, "Tes

train_preds = model.predict_from_sequences(train_smiles[:5])
for true, pred in zip(train_smiles[:5], train_preds):
    pred_str = "".join(pred).strip()
    print("TRUE:", true)
    print("PRED:", pred_str)
    print("MATCH:", pred_str == true)
    print()
```

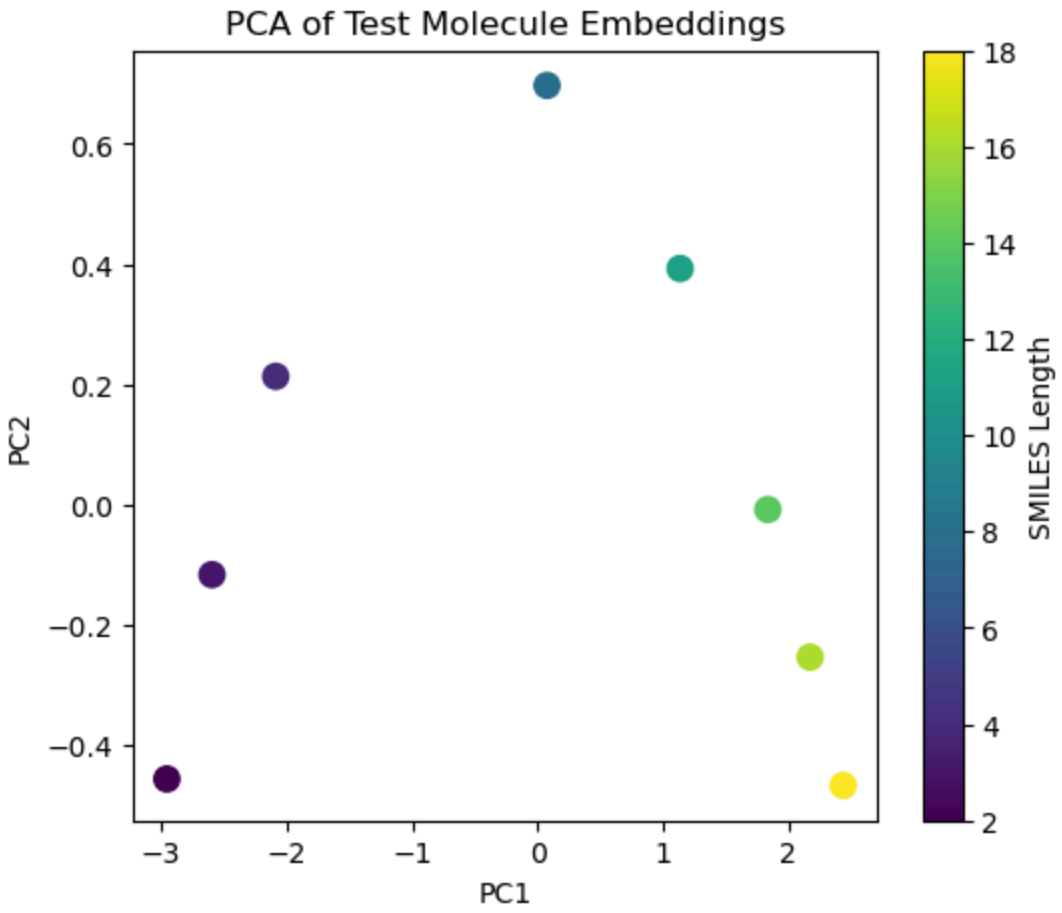
```
Train total data points: 10
Train correct regenerations: 1
Train accuracy: 10.00%

Test total data points: 8
Test correct regenerations: 1
Test accuracy: 12.50%

TRUE: CCCCCCCCCO
PRED: CCCCCCCCCCO
MATCH: False

TRUE: CCCCCCCCCCCCCCCCCCO
PRED: CCCCCCCCCCCCCCCCCC
MATCH: False

TRUE: CCCCCCCCCCCCCCCCO
PRED: CCCCCCCCCCCCCCCCCO
MATCH: False

TRUE: CCCCCO
PRED: CCCCCOO
MATCH: False

TRUE: CCCCCCCCCCCCO
PRED: CCCCCCCCCCCCCO
MATCH: False
```

3.4 Visualize molecule embeddings

❓ 3.4.1 Visualize the embeddings of the test dataset in a 2D scatter plot using PCA, and color the points based on the length of the corresponding SMILES.

Useful link: predict_embeddings

In [51]:
```python
# write your code below
from sklearn.decomposition import PCA
import matplotlib.pyplot as plt
import numpy as np

# get embeddings for the test SMILES
embeddings = model.predict_embeddings(test_smiles)
embeddings = np.array(embeddings)

# reduce to 2D with PCA
pca = PCA(n_components=2)
embeddings_2d = pca.fit_transform(embeddings)

# color by SMILES length
lengths = [len(s) for s in test_smiles]

plt.figure(figsize=(6, 5))
scatter = plt.scatter(
    embeddings_2d[:, 0],
    embeddings_2d[:, 1],
    c=lengths,
    cmap="viridis",
    s=80
)

plt.xlabel("PC1")
plt.ylabel("PC2")
plt.title("PCA of Test Molecule Embeddings")
plt.colorbar(scatter, label="SMILES Length")
plt.show()
```



❓ 3.4.2 Describe your observations and explain whether you believe this embedding can be used as "molecular features." *Consider your answer from 3.1.2. What has our model learned from this training data? Is this embedding meaningful? Is this embedding generalizable?*

✅ Answers: The PCA plot suggests that the embeddings are mainly capturing simple differences in the SMILES strings, especially their length. Points with shorter SMILES appear on one side of the plot, while longer SMILES tend to appear on the other side, which suggests the model has learned patterns related to chain length more than deeper chemical properties. This makes sense because the dataset consists of very similar alcohol-like molecules, mostly differing by the number of repeated carbon atoms before the oxygen. Therefore, the embedding is somewhat meaningful for this specific dataset, since it reflects a real structural pattern, but it is limited as a general molecular feature representation. It likely would not generalize well to more diverse molecules because the training data are too small and too simple, so the model has mostly learned sequence length and repetitive token structure rather than broad chemical features.